Agenda

1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Von-Neumann



John von-Neumann (eigentlich *Neumann János Lajos), *1903 in Budapest,* †1957 in Washington, DC.

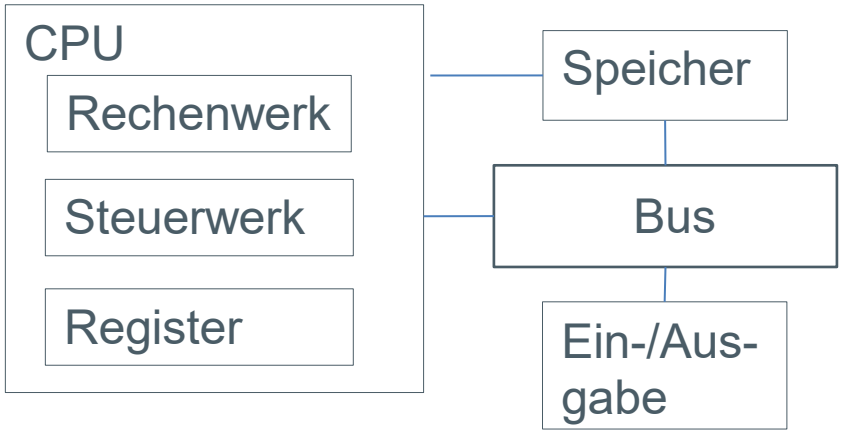
- Promoviert 1926 in Budapest, habilitiert in Berlin (1927)
- Ab 1943 Mitarbeiter am Manhattan-Projekt
- Zusammen mit den ungarisch-stämmigen Leo Szilard, Edward Teller, George Polya und Eugene Wigner gehörte er zu den „Marsianern“.
- Beschrieb 1945 die Architektur der ersten Computer des ENIAC und EDVAC, die nach dem Prinzip, das heute als von-Neumann-Architektur bekannt ist, arbeiteten.
- Von-Neumann-Architektur: „Programme und Daten sind im gleichen Speicher“

Rechnerarchitektur

*„**Computer Architecture** = Instruction Set Architecture + Computer Organization + Hardware“*

- **Instruction Set Architecture** (ISA)
  - Die Menge aller Kommandos und Anweisungen, die die CPU versteht
  - Je größer das ISA, desto komplexer (mehr Schaltkreise) wird die CPU
  - Beispiel: Intel x64 besitzt > 1000 Instruktionen

- **Computer Organization**
  - Funktionale Sicht auf den Aufbau eines Computers
  - Die meisten Computer basieren auf der von-Neumann-Architektur
  - CPU holt Instruktionen aus dem Speicher (**fetch-decode-execute**)
  - Im Speicher sind Daten und Programmcode
  - Der Bus (die Busse) übertragt Daten zwischen CPU, Speicher und IO



Konsequenzen der Von-Neumann-Architektur

- Die Struktur des Computers ist unabhängig vom bearbeiteten Problem. Der Computer ist **programmierbar**.
- Anweisungen und Operanden (einschl. Zwischenergebnissen) werden in demselben physikalischen Speicher gespeichert.
- Der Speicher wird in Zellen gleicher Größe geteilt. Die Zellennummern heißen **Adressen**.
- Das Programm besteht aus einer Folge von elementaren Befehlen, die in der Reihenfolge der Speicherung bearbeitet werden.
- Abweichungen von der Reihenfolge sind mit (bedingten oder unbedingten) **Sprungbefehlen** möglich.

```
_start:
   movl $1, %eax    # Lade 1 in Register EAX
   addl $2, %eax    # Addiere 2 zu EAX
   jmp Label        # Springe zu „Label“
…
Label:
   …
```

RISC vs. CISC

- RISC = **„Reduced Instruction Set Computer“** (z.B. ARM, Apple M3)
  Wenige, einfache Befehle
  Schnell, weil einfache Befehle schnell ausgeführt werden können.
- CISC = **„Complex Instruction Set Computer“** (z.B. x86)
  Viele komplexe Befehle.
  Schnell, weil nur wenige Befehle für ein Programm notwendig sind.

Performance eines Rechners wird bestimmt durch:

$$\frac{time}{program} = \frac{time}{cycle} * \frac{cycles}{instruction} * \frac{instructions}{program}$$

CISC optimiert $\frac{instructions}{program}$, RISC optimiert $\frac{cycles}{instruction}$

Agenda

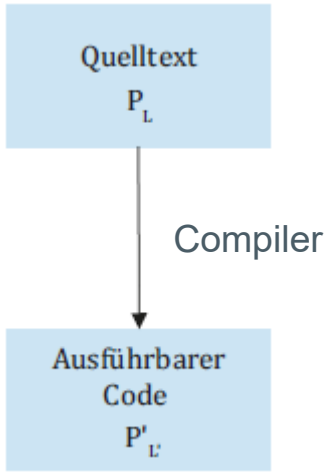
1.	Einleitung	2
2.	Versionsverwaltung	17
3.	Betriebssysteme	48
a.	Dateisystem	69
b.	Prozesse	77
c.	Speicherverwaltung	86
d.	Virtualisierung und Containerisierung	95
4.	Verteilte Systeme	102
5.	REST-Services	134
6.	Datenbanken	203
a.	Datenmodellierung	216
b.	Relationenmodell	236
c.	SQL	260
d.	JDBC	303
7.	Rechnerarchitektur	321
8.	Compiler	327
9.	IT-Security	343

Was ist ein Compiler?

Compiler

Compilare (lat.): Aufhäufen.

Ein Computerprogramm, das Quellcodes einer Programmiersprache übersetzt, so dass das Ergebnis auf dem Computer ausgeführt werden kann.



Compilerbau

Die Informatik-Disziplin, die sich mit dem Entwurf und der Programmierung von Compilern befasst.

Beispiele: Java Java Bytecode
TypeScript JavaScript
C Maschinensprache

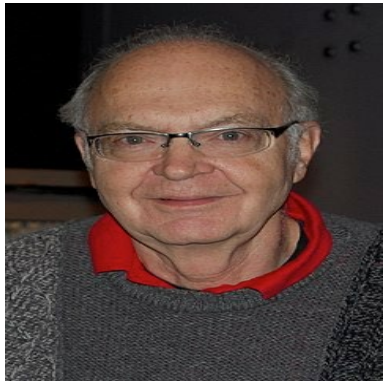
Historie



Grace M. Hopper
Entwickelte 1952 den A-0 Compiler und 1960 den ersten Cobol Compiler.



John Backus
Entwickelte 1957 bei IBM den ersten FORTRAN Compiler.
Aufwand damals: 54 Mannjahre!
Turing Award 1977

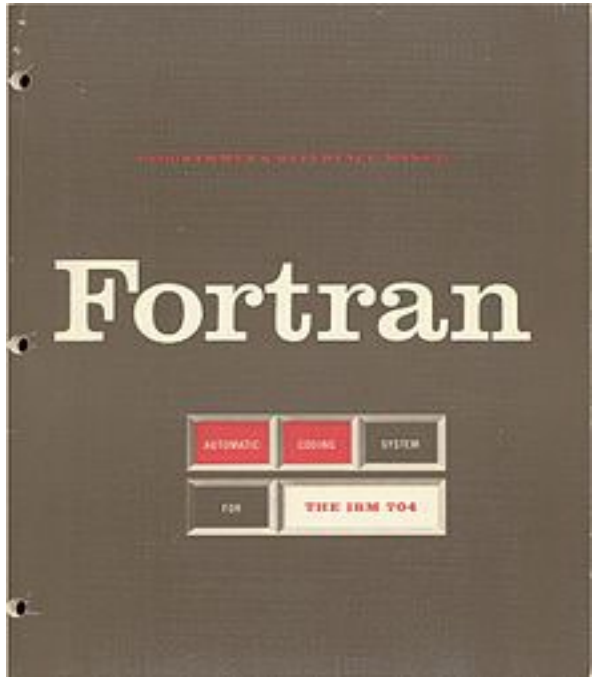


Donald Knuth
Entwickelte in den 1960‘er Jahren die Theorie der Syntaxanalyse.
Turing Award 1974

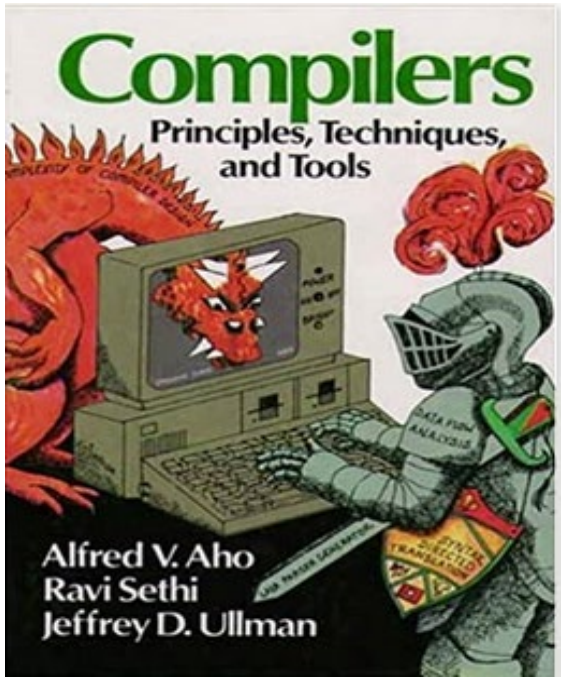


Niklaus Wirth
Entwickelte mehrere Programmier-sprachen (u.a. Pascal).
Turing Award 1984

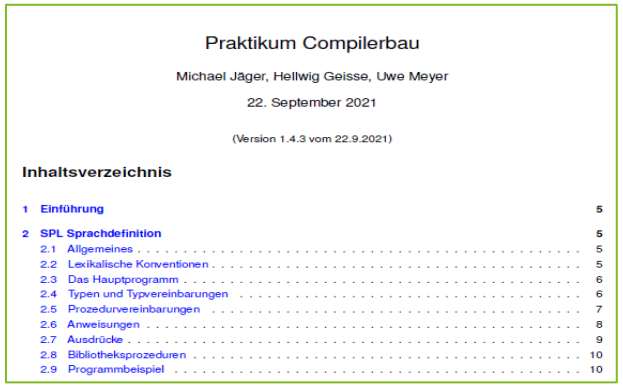
Vom Wert der Theorie …



1956: Keine Theorie → 54 Mannjahre Aufwand



1985: Der Compilerbauer kämpft mit den Waffen der theoretischen Informatik gegen die Komplexität des Compilers.



Praktikum Compilerbau

Michael Jäger, Hellwig Geisse, Uwe Meyer

22. September 2021

(Version 1.4.3 vom 22.9.2021)

Inhaltsverzeichnis

1 Einführung 5
2 SPL Sprachdefinition 5
2.1 Allgemeines . . . 5
2.2 Lexikalische Konventionen . . . 5
2.3 Das Hauptprogramm . . . 6
2.4 Typen und Typvereinbarungen . . . 6
2.5 Prozedurvereinbarungen . . . 7
2.6 Anweisungen . . . 8
2.7 Ausdrücke . . . 9
2.8 Bibliotheksprozeduren . . . 10
2.9 Programmbeispiel . . . 10



Bison

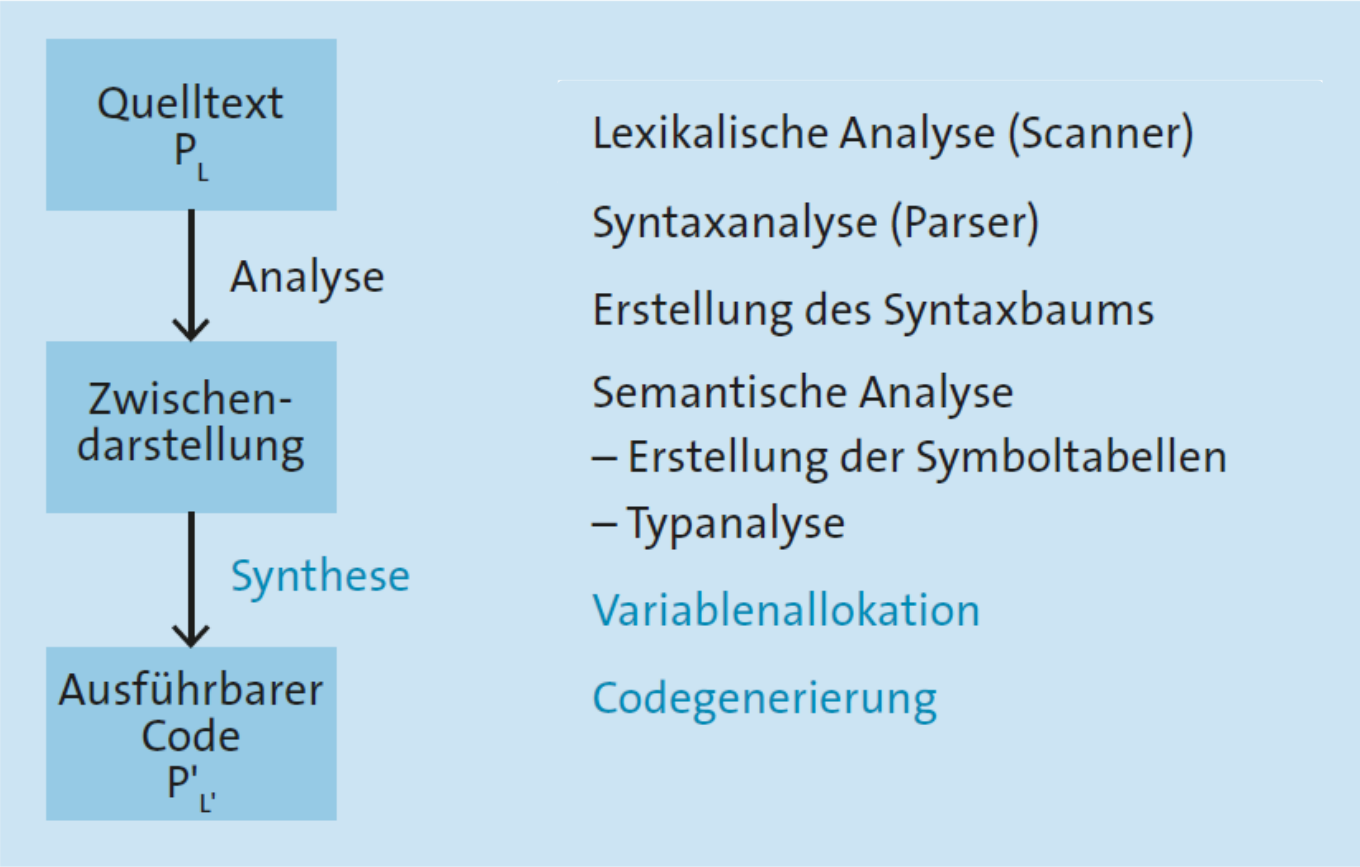
JFlex

CUP LALR Parser Generator for Java™

2024: Studierende der THM entwickeln mit modernen Tools einen SPL- Compiler in 3 Monaten.

Arbeitsweise eines Compilers - Phasenmodell

Typischerweise wird die Arbeit eines Compilers in mehreren Phasen durchgeführt



Beispiel

```
publik class Pi2CBExample {
  private int counter = 0;

  public static void main(String[] args)
{
    int i = i  §  54321;
    int j = i+((1);
    System.out.println(counterr);
    j = "Hallo" + i;
  }
}
```

Welche Fehler sollte der Java-Compiler hier erkennen?

Beispiel

Quelltext

```
var  x :  int ;
var  y :  int ;
…
x := y + 3;
```

Ergebnis der lexikalischen Analyse

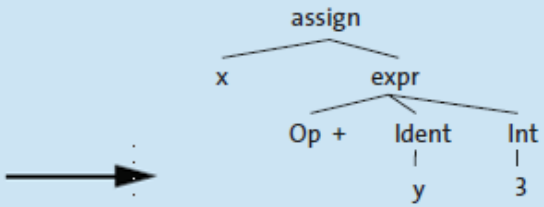
VAR, IDENT(x), COLON, IDENT (int), SEMIC

VAR, IDENT(y), COLON, IDENT (int), SEMIC

...

IDENT(x), ASGN, IDENT(y), PLUS, INTLIT(3), SEMIC

Syntaxbaum (Ausschnitt)



Symboltabelle (Ausschnitt)

Symbol	Typ
x	int
y	int

Variablenallokation

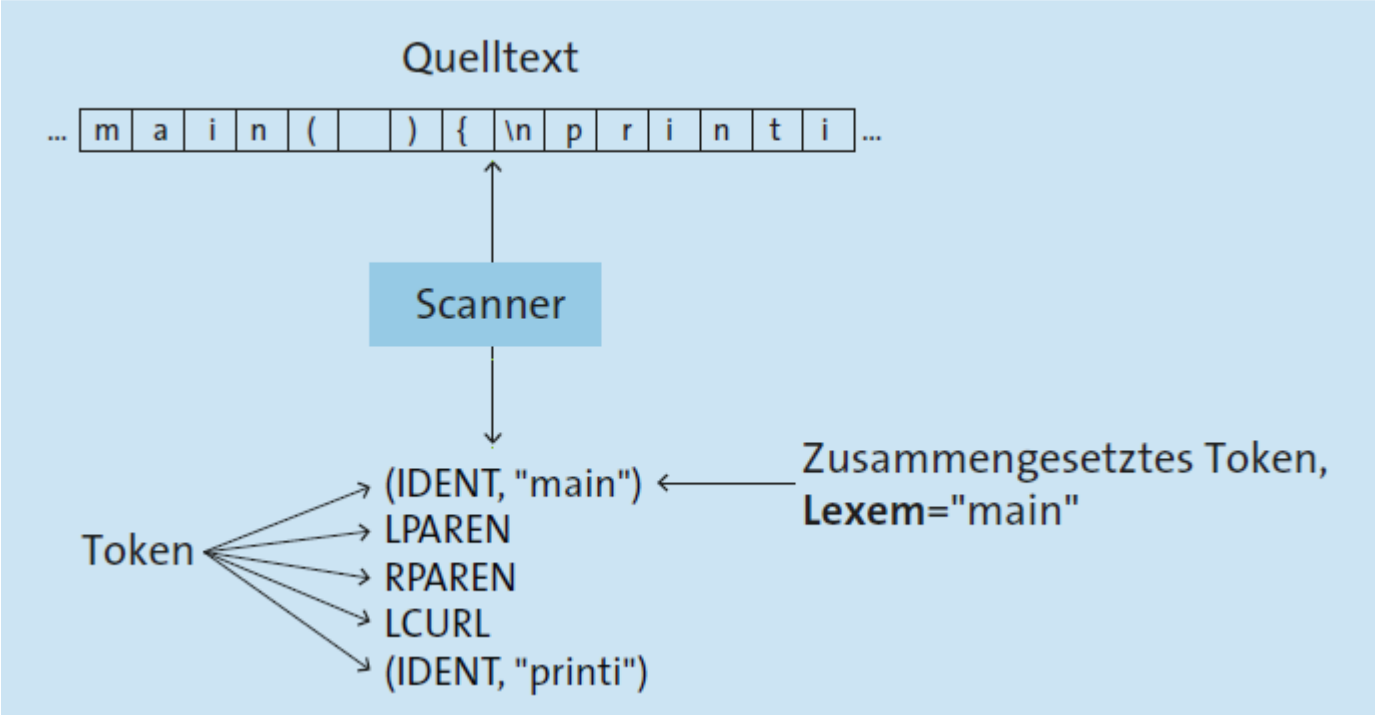
Variable	Speicherplatz
x	FP-4
y	FP-8

Assemblercode

```
add $8, $25, -4
ldw $9, $25, -8
add $9, $9, 1
stw $9, $8, 0
```

Lexikalische Analyse

Die Aufgabe der Lexikalischen Analyse ist es, die Zeichenkette des Quelltextes in ***Tokens*** zu zerlegen, sodass wir danach die grammatikalische Korrektheit prüfen können.
Der **Scanner** ist das von uns zu erstellende Programm, das die lexikalische Analyse durchführt.



Tokens
=
lexikalische Grundeinheiten

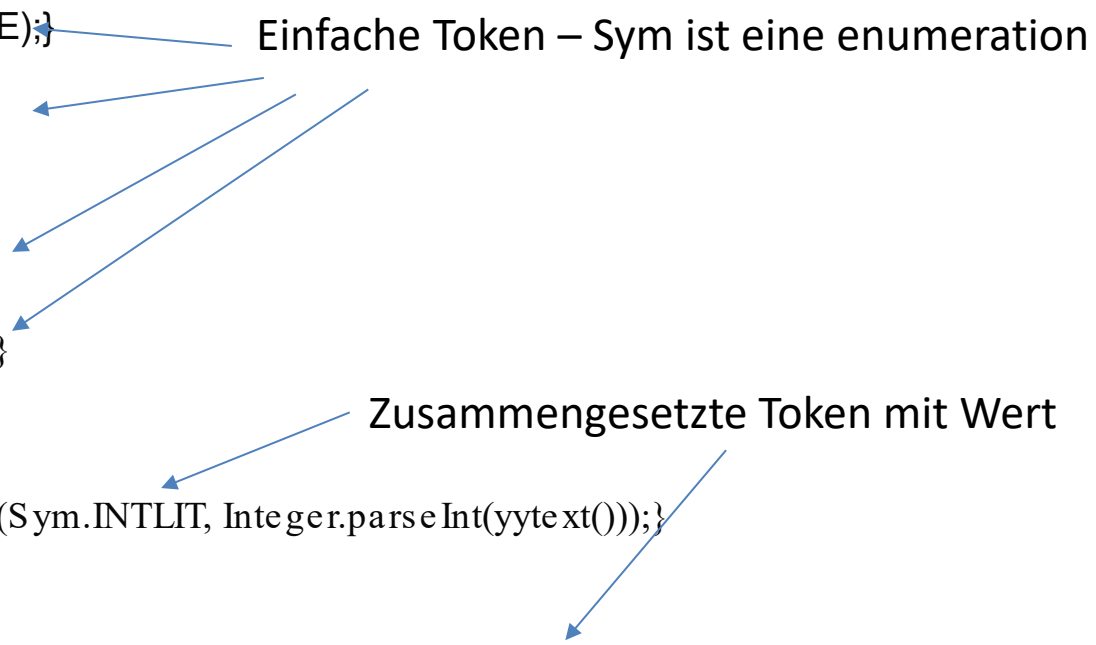
Scanner
=
Programm, das die lexikalische Analyse durchführt

Lexem
=
erkannter Text

Lexikalische Analyse – Reguläre Ausdrücke -> Token

Statt die lexikalische Analyse zu programmieren, kann man die lexikalischen Elemente als reguläre Ausdrücke angeben:

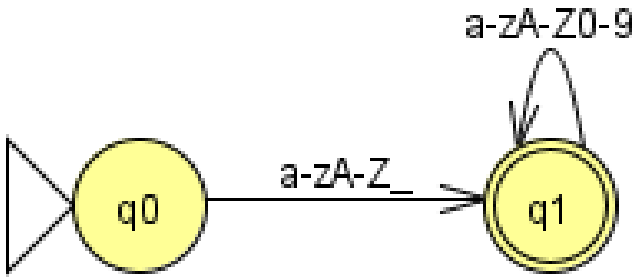
- Schlüsselwörter:
  - type {return symbol(Sym.TYPE);}
  - if {return symbol(Sym.IF);}
- Sonderzeichen
  - = *{return symbol(Sym.EQ);}*
  - ; {return symbol(Sym.SEMIC);}
- Zahlen
  - [0-9]+ {return symbol(Sym.INTLIT, Integer.parseInt(yytext()));}
- Bezeichner
  - [a-zA-Z_][a-zA-Z_0-9]* {return symbol(Sym.IDENT, new Identifier(yytext()));}



Lexikalische Analyse – Umsetzung

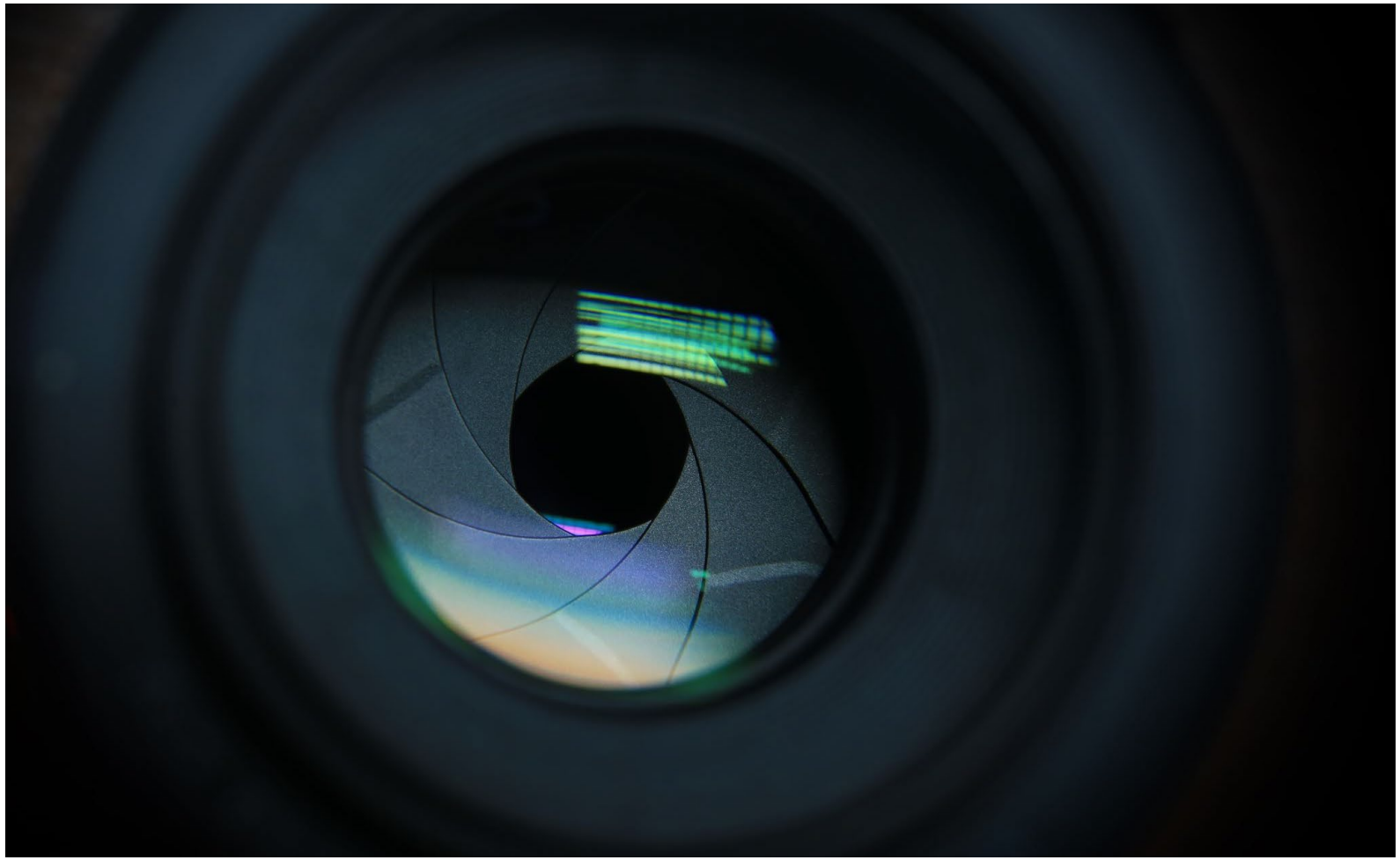
Der Scanner simuliert einen endlichen Automaten!

Beispielsweise erkennt der u.a. Automat einen Bezeichner:



Ein Scanner-Generator liest die Spezifikation der regulären Ausdrücke und generiert daraus den Automaten.

Demo



Syntaktische Analyse

Die syntaktische Analyse nimmt als Eingabe die Liste der erkannten Tokens.

Eine (kontextfreie) Grammatik beschreibt, welche Konstrukte in einer Sprache erlaubt sind.

Beispiel einer ganz einfachen Sprache:

$$program \rightarrow expression$$
$$expression \rightarrow term \mid expression + term$$
$$term \rightarrow factor \mid term * factor$$
$$factor \rightarrow (\ expression\ ) \mid number \mid variable$$
$$number \rightarrow 0 \mid 1 \ \ldots \mid 9$$
$$variable \rightarrow a \mid \ldots \mid z$$

Aufgabe:

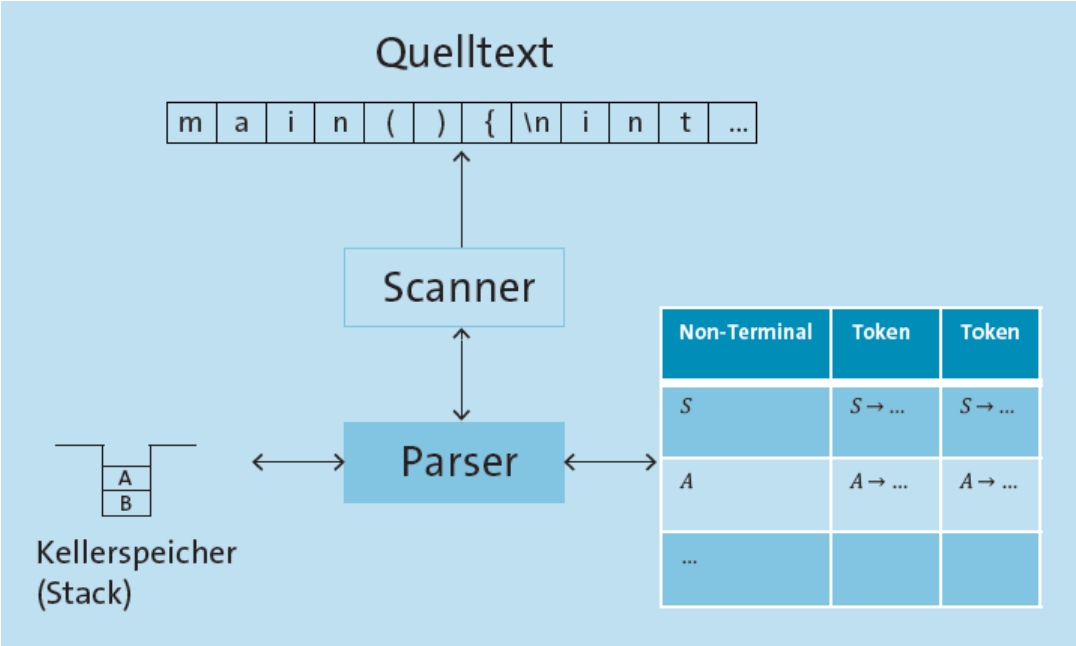
Generieren Sie ein Programm!

z.B.: $program \Rightarrow\ expression \Rightarrow expression + term \Rightarrow factor + term \Rightarrow variable + term$
$\Rightarrow x + term \Rightarrow x + term * factor \Rightarrow \cdots \Rightarrow x + 3 * 2$

Syntaktische Analyse - Kellerautomat

Ein endlicher Automat kann nicht alle kontextfreien Grammatiken erkennen, da er außer dem Zustand keinen Speicher hat.

Verwendet werden Kellerautomaten, die einen Keller (Stack) benutzen.



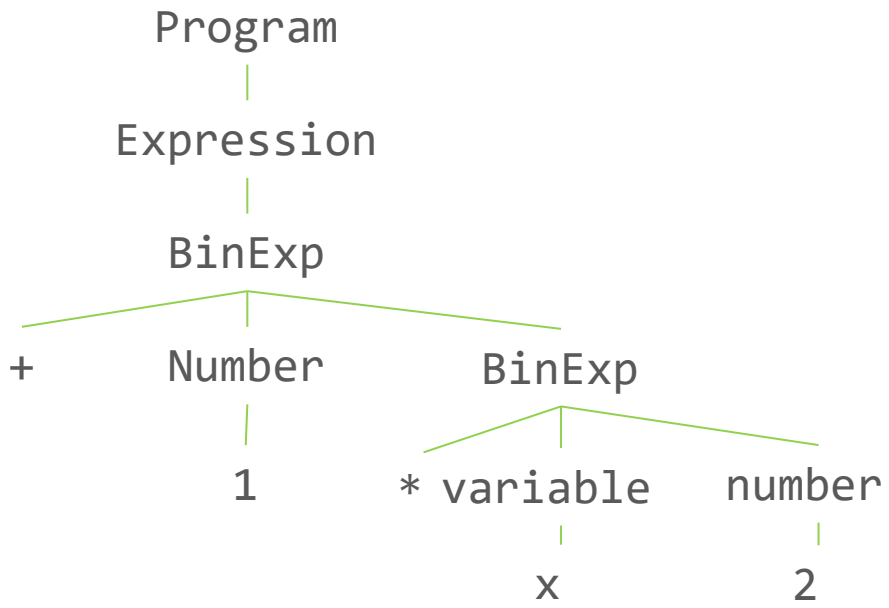
Parsergeneratoren lesen die Grammatik der Programmiersprache ein und erzeugen einen Kellerautomaten („Parser“), der wiederum den Scanner verwendet.

Syntaktische Analyse - Abstrakter Syntaxbaum

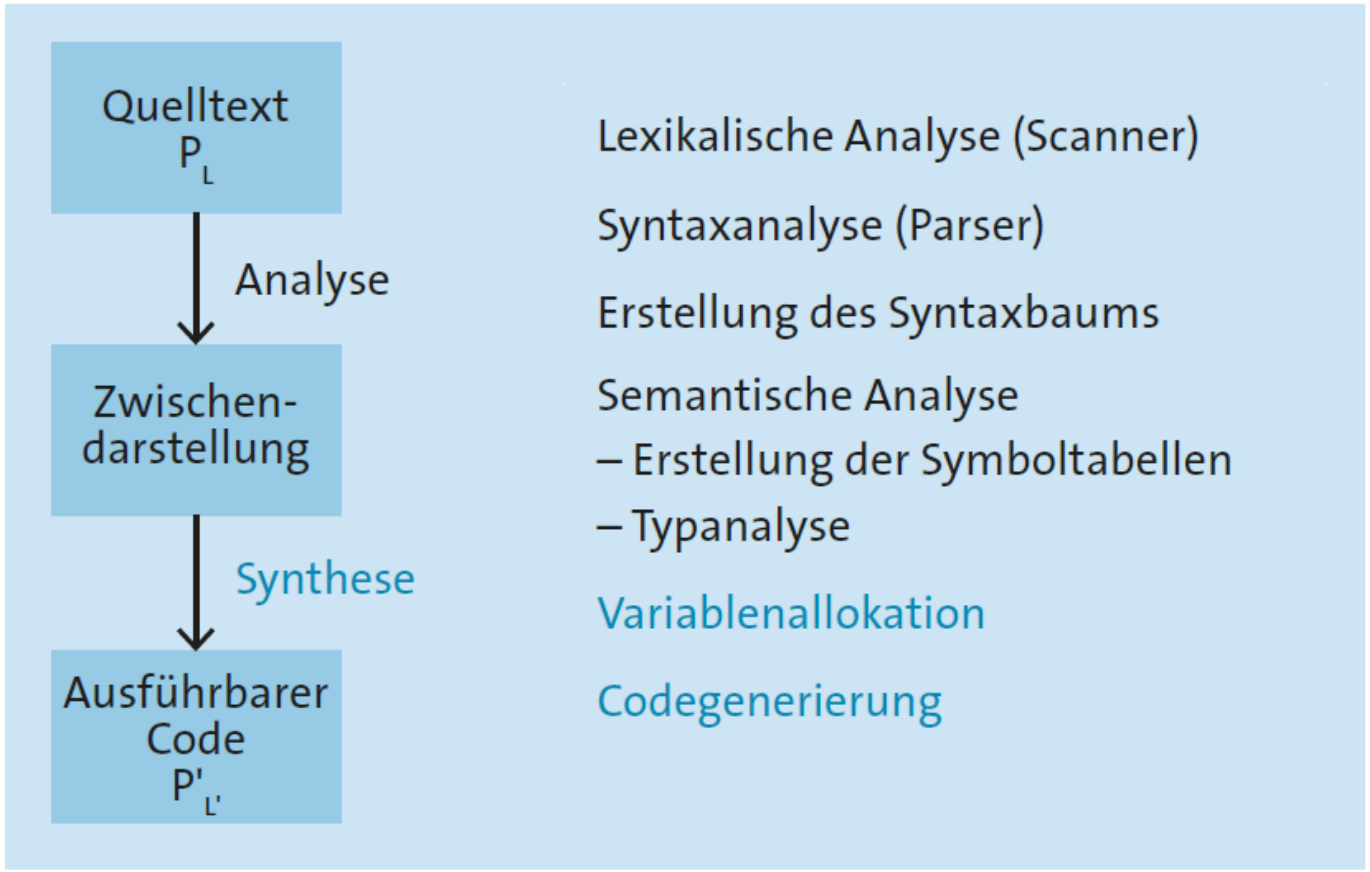
Als Datenstruktur, um mit dem Programm weiterarbeiten zu können, wird üblicherweise ein Baum verwendet. Dieser „abstrakte Syntaxbaum“ enthält alle wesentlichen Teile des Quellprogramms.

$1 + x * 2$





Semantische Analyse und Synthese



Auch wenn das Programm grammatikalisch korrekt ist, könnten noch Fehler enthalten sein (siehe Folie 329). Alle diese Fehler muss der Compiler erkennen.

Zum Schluss wird z.B. Assemblercode generiert.

Ausblick auf „Automaten, Sprachen und Compiler“

- Die Vorlesung „Automaten, Sprachen und Compiler“ setzt auf Theoretische Informatik I und II auf und behandelt die Verfahren und Grundlagen der Syntaxanalyse und der Codegenerierung.
- Im Praktikum entwickeln Sie Teile eines Scanners, eines Parsers und eine einfache Codegenerierung für eine einfache Programmiersprache.
- Im Wahlpflichtmodul „Compilerbau“ entwickeln Sie einen vollständigen und funktionsfähigen Compiler für eine kleine Programmiersprache.